

# FSM 리팩토링 변경 로그

Player 직접 입력 기반 FSM | 구조 변화와 검증 이력

## 목적

기능 명세가 아닌 실제 리팩토링의 판단, 책임 이동, 검증 상태를 시간순으로 기록한다.

# 1. 리팩토링 이전 구조

초기 FSM은 State를 시작 시점에 생성해 Dictionary에 보관하고 전환 시 재사용했다. 이는 State 전환마다 객체를 생성하는 방식의 GC와 전환 비용을 피하기 위한 선택이었다.

당시 `AgentController`는 State 생성·보관·전환과 이동, 입력, 전투, Animator 접근을 함께 제공했다. State 기반은 `AgentStateBase<T>`와 `GroundedAgentStateBase`로 계층화되어 있었고, `AgentStateMachine<IAgentState>`가 현재 State와 전환을 관리했다.

State는 `\_agent.ChangeState(...)`를 직접 호출했다. 따라서 전이 조건, 전환 실행, Controller 접근 경로가 하나의 State 안에서 결합되었고, 기능 구현을 Handler로 분리한 뒤에도 실제 의존성이 잘 드러나지 않았다.

| 구분         | 리팩토링 전                                      | 핵심 문제                                 |
|------------|---------------------------------------------|---------------------------------------|
| 상태 기반      | `AgentStateBase` + `GroundedAgentStateBase` | generic/지상 전용 기반이 State 구현에 전파        |
| 상태 전환      | State 내부에서 `_agent.ChangeState` 직접 호출       | 전이 조건과 전환 실행이 결합                      |
| 상태 머신      | `AgentStateMachine`                         | Controller-StateMachine-State에 책임이 분산 |
| Controller | 기능 접근과 State 관리를 폭넓게 담당                     | State별 필요 기능이 누적되어 God Object 성격 확대   |

## 초기 흐름

State → AgentController → Input / Movement / Combat / Animator

State → \_agent.ChangeState → AgentStateMachine

## 2. 첫 구조 전환

첫 변경은 단순히 Factory를 추가한 작업이 아니었다. State 기반, 전환 책임, 상태 머신 소유권을 먼저 단순화하고, 이후 Factory가 필요한 의존성과 전이 규칙을 조합하도록 바꾼 단계였다.

| 구분    | 리팩토링 전                                         | 1차 변경                                       | 상태    |
|-------|------------------------------------------------|---------------------------------------------|-------|
| 상태 기반 | `AgentStateBase` +<br>`GroundedAgentStateBase` | 비제네릭 `AgentStateBase` 하나로 통합                | 적용    |
| 상태 전환 | State에서 `_agent.ChangeState` 직접 호출             | `ITransitionRule` 순차 평가 후 `OnTransition` 발행 | 부분 적용 |
| 상태 머신 | `AgentStateMachine`                            | `AgentController`가 `_currentState` 직접 보유    | 적용    |

### 전환 책임의 변화

`AgentStateBase` 통합으로 공통 생명주기와 전이 규칙 등록 지점을 하나로 모았다. `AgentStateMachine` 제거 후에는 Controller가 State Dictionary와 `\_currentState`를 직접 소유해 현재 State의 소유자와 전환 실행 지점을 명확히 했다.

`ITransitionRule`은 State가 전환을 직접 실행하는 대신 전환 가능 여부와 다음 State를 선언한다. State는 Rule을 순차 평가하고 `OnTransition`을 발행하며, Factory가 이를 Controller의 전환 처리와 연결한다. Player FSM에는 적용됐지만 Monster와 기존 State 이행은 남아 있어 부분 적용으로 기록한다.

Controller → StateFactory → State(필요한 Handler / Input / Animator capability만 주입)

ITransitionRule → State.OnTransition → AgentController 전환

### 3. 후속 구조 개선 이력

| 단계 | 변경                       | 문제 / 이유                    | 결과                                 |
|----|--------------------------|----------------------------|------------------------------------|
| 01 | State 기반·상태 머신 단순화       | 기반 클래스와 StateMachine 책임 분산 | 비제네릭 StateBase와 Controller 직접 관리   |
| 02 | StateFactory 의존성 주입      | Controller가 모든 State 기능 제공 | State별 의존성이 생성자에 드러남               |
| 03 | ITransitionRule 도입       | 전이 조건·이벤트가 State 직접 호출과 결합 | Factory에서 Rule 조합, Player 기준 부분 적용 |
| 04 | Type 기반 State 관리         | 공용 StateType enum 누적       | 상태 확장 간 충돌 부담 완화                   |
| 05 | PlayerInput UniRx 정리     | 구독 없는 입력까지 반응형 값           | 홀드·소비형 요청·이벤트로 분리                  |
| 06 | GroundedState 통합         | Idle/Move FSM 왕복           | Blend Tree로 표현 이관                  |
| 07 | Animator 전이 계약           | FSM/Animator 상태 불일치        | IsGrounded와 종료 목적지 정리              |
| 08 | 점프 경계 보강                 | 가장자리·재입력 재상승               | 점프 소비와 하강 Fall 조건                  |
| 09 | Agent/Player Animator 분리 | 공통 Animator에 Player 값 누적   | 공통 capability와 Player 전용 API 분리    |
| 10 | Prefab Animator 바인딩      | Root/Hero Animator 이중 구성   | 실제 Hero child Animator 제어          |

### 4. 현재 구조와 검증

PlayerStateFactory는 Grounded, Jump, Fall, Attack, Hit, Death를 생성하고 전이 Rule을 조합한다. Grounded는 MoveSpeed Blend Tree로 정지/이동만 표현하며, 점프·낙하·공격·피격·사망만 별도 FSM State로 유지한다.

| 항목                                          | 상태    | 남은 확인                            |
|---------------------------------------------|-------|----------------------------------|
| C# 컴파일 / Animator parameter 계약 / Prefab 바인딩 | 구성 완료 | 수동 Play Mode에서 실제 동작 확인          |
| 점프 경계 체감                                    | 미완료   | 가장자리 접근·공중 재입력·홀드 반복             |
| 공격 홀드 동기화                                   | 미완료   | Grounded/Fall 공격 반복과 종료 복귀       |
| Monster 이행                                  | 보류    | 전용 capability·Factory·SO 설계 후 진행 |